Étude d'opportunité et de faisabilité — Stockage objet MinIO / S3 — FileFlow

Titre	Étude — Option de stockage des données sur MinIO / S3 (activable) en alternative à `ROOT_DIR`
Version	1.0
Date	22 juillet 2026
Statut	Pour décision
Propriétaire	Driss OUAAZIZ — Software Architect
Classification	Interne

> Nature du document : étude d'aide à la décision. Aucune modification de code n'a été réalisée. La décision de mise en œuvre est laissée en suspens.

Table des matières

1. Objet et contexte
2. Synthèse et recommandation
3. Utilité
4. Faisabilité technique
5. Impacts transverses
6. Approche recommandée en cas de décision favorable
7. Solution intermédiaire sans développement
8. Options de décision
9. Références

1. Objet et contexte

FileFlow stocke aujourd'hui les fichiers utilisateurs et ses données internes (versions, corbeille, uploads en cours, métadonnées) sur le **système de fichiers local**, sous le répertoire racine `ROOT_DIR`. Le store de configuration (comptes, partages, favoris…) est, lui, tenu dans une base **SQLite** distincte (`CONFIG_DIR`).

La présente étude évalue **l'utilité** et la **faisabilité** d'ajouter une **option activable** permettant de stocker les fichiers sur un **service de stockage objet compatible S3 (MinIO)**, en alternative à `ROOT_DIR`, sans supprimer le mode fichier existant.

Périmètre de l'étude : les données de fichiers (contenu, arborescence, versions, corbeille, uploads). **Hors périmètre :** le store de configuration SQLite, qui fait l'objet d'une remarque spécifique (§3).

2. Synthèse et recommandation

Verdict : faisable, mais à ne prioriser que si la feuille de route vise réellement le passage à l'échelle ou la durabilité multi-nœuds. Pour le positionnement actuel du produit (auto-hébergement simple, mono-binaire), le rapport coût/bénéfice est **défavorable** et l'option est à **différer**. Si elle est retenue, elle doit être réalisée

comme **backend de stockage alternatif activable**, derrière une abstraction, et **jamais** en remplacement pur et simple de `ROOT_DIR`.

La dépendance au système de fichiers est **concentrée** : environ 96 appels répartis sur **trois modules** seulement (`files.rs`, `uploads.rs`, `upload_requests.rs`) ; les stores de configuration n'y touchent pas. La refonte est donc circonscrite, mais elle comporte quelques **points durs** structurants (répertoires, renommage, upload résumable).

3. Utilité

3.1 Bénéfices

BÉNÉFICE	DESCRIPTION
Capacité	Dépasse la limite d'un volume `ROOT_DIR` unique ; volumétrie quasi illimitée
Durabilité	MinIO (codage à effacement, multi-nœuds) offre une meilleure résistance à la perte disque qu'un volume local unique
Découplage calcul / stockage	Pertinent en orchestration (Kubernetes) : les instances applicatives n'ont plus besoin d'un volume persistant pour les fichiers
Consolidation	Réutilisation d'une infrastructure MinIO / S3 déjà exploitée par l'organisation

3.2 Limites et tensions

- **Contradiction avec le positionnement produit.** La note de cadrage (`01-note-de-cadrage.md`) met en avant « mono-binaire, simplicité d'exploitation, **sans dépendance externe lourde** ». MinIO est un service supplémentaire à déployer, superviser et sauvegarder : pour un petit déploiement, la complexité ajoutée n'est pas compensée par un gain.
- **Gain d'échelle partiel.** Déporter les fichiers sur l'objet **ne rend pas FileFlow scalable horizontalement** : l'état de configuration reste en **SQLite mono-connexion sérialisée** (voir registre de dette, entrée D8). Une haute disponibilité réelle imposerait d'externaliser également ce store. Seul, le stockage objet apporte **capacité et durabilité**, pas la **haute disponibilité**.
- **Souveraineté : neutre.** MinIO étant auto-hébergeable, le discours de souveraineté du produit demeure valable.

4. Faisabilité technique

4.1 Cartographie de la dépendance au système de fichiers

MODULE	APPELS SYSTÈME DE FICHIERS	RÔLE
`files.rs`	~74	Cœur des opérations fichiers (listing, download, upload, rename, move, delete, versions, recherche, tags)
`uploads.rs`	~15	Upload chunké résumable (`.part`, écriture à offset, `rename`)
`upload_requests.rs`	~7	Upload public anonyme
Autres stores	0	Déjà sur SQLite — non concernés

La refonte est donc **circonscrite à trois modules**.

4.2 Ce qui se porte bien sur le stockage objet

FONCTION	CORRESPONDANCE S3 / MINIO
Téléchargement / prévisualisation avec plage `Range` (RFC 7233)	`GET` avec en-tête `Range` — supporté nativement
Taille et date de modification	`ContentLength`, `LastModified`, `ETag`
Listing d'un dossier	`ListObjectsV2` par préfixe (paginé)
Application des quotas (`used_bytes`)	Déjà tenue en base, indépendante du backend
Téléchargement ZIP groupé	`GET` par objet, mise en flux vers l'archive
Confinement des chemins	Plus simple : ni symlink ni traversée FS — validation/assainissement de clé

4.3 Points durs nécessitant une refonte

POINT DUR	DIFFICULTÉ SUR L'OBJET	TRAITEMENT NÉCESSAIRE
Répertoires (mkdir, `is_dir`, dossiers vides)	Pas de vrais dossiers, uniquement des préfixes	Émulation par convention de préfixe + objets marqueurs pour les dossiers vides
Renommage / déplacement (10 sites)	Pas de renommage : `copy` + `delete`, **O(n)** et non atomique pour un dossier	Réécriture de la logique ; acceptation de la copie serveur-à-serveur
Upload chunké résumable (écriture à offset + `.part` + `rename`)	Pas d'écriture à accès direct	Bascule vers le **multipart upload** S3 (parties ≥ 5 Mio) ou mise en tampon local temporaire
Corbeille et versions (`rename` dans `.fileflow/`)	idem renommage	`copy` vers préfixe `trash/` ou `versions/` puis `delete`
Recherche plein-texte (`WalkDir` + lecture)	Listing acceptable, mais lecture = `GET` par objet	Coût réseau accru ; bornage plus strict

5. Impacts transverses

- **Build et sécurité (tension forte).** Un client S3 avec TLS réintroduit une pile de dépendances (HTTP, TLS) qui met en question l'acquis « binaire statique musl, image *distroless* sans vulnérabilité de socle » (voir ADR-0003 et ADR-0004). À mitiger par un **client entièrement en Rust** (avec `rustls`) afin de préserver la compilation statique ; l'option **augmente néanmoins la surface de dépendances et de CVE**.
- **Cohérence.** MinIO est **fortement cohérent** (lecture après écriture) : ce n'est plus un obstacle.
- **Performance.** Chaque opération devient un appel réseau. Les opérations « dossier » (renommage, déplacement, suppression récursive) passent de O(1) local à **O(n) copies**.
- **Effort estimé. Moyen à élevé** (plusieurs semaines pour une version robuste et testée), l'essentiel portant sur l'émulation des répertoires/renommage et la refonte de l'upload résumable. À accompagner d'un ADR, de tests, et d'une mise à jour du DAT et du dossier d'exploitation.

6. Approche recommandée en cas de décision favorable

1. **Introduire un trait d'abstraction** `Storage` (`open_range`, `put`, `delete`, `list`, `copy`, `exists`, `stat` …) avec deux implémentations : `LocalFs` (comportement actuel, inchangé) et `S3` (MinIO).

2. **Activation par variable d'environnement** (`STORAGE_BACKEND=fs|s3` , plus `S3_ENDPOINT` , `S3_BUCKET` , `S3_ACCESS_KEY` , `S3_SECRET_KEY`), **`fs` par défaut** — garantissant **aucune régression** pour les déploiements existants.
3. **Concentrer la refonte** sur `files.rs` , `uploads.rs` et `upload_requests.rs` ; les stores de configuration restent inchangés.
4. **Phaser la livraison :**
    - P1 — Trait `Storage` + backend `fs` iso-comportement (aucune régression).
    - P2 — Backend S3 : lecture, écriture simple, listing, métadonnées.
    - P3 — Renommage / déplacement / corbeille / versions (copie).
    - P4 — Upload multipart + recherche.

7. Solution intermédiaire sans développement

Si le besoin immédiat est uniquement la **capacité de stockage**, il est possible d'**évaluer** l'objet sans aucun développement : faire pointer `ROOT_DIR` vers un **point de montage adossé à un stockage objet** (passerelle de type système de fichiers sur S3, ou volume CSI). Réserves : performances dégradées sur les opérations de métadonnées et de renommage, et sémantique de cohérence à valider selon la passerelle. Cette approche convient à une évaluation, pas à une cible de production exigeante.

8. Options de décision

OPTION	QUAND LA RETENIR	CONSÉQUENCE
A — Statu quo (`ROOT_DIR` uniquement)	Positionnement self-host simple maintenu	Aucun coût ; capacité liée au volume
B — Différer (consigner au registre de dette / feuille de route)	Besoin non avéré à court terme	Décision réévaluée quand un moteur d'échelle apparaît
C — Mettre en œuvre (backend S3 activable)	Cible cloud-native / multi-instances confirmée	Effort moyen à élevé ; à mener avec l'externalisation du store de configuration

Recommandation : option B en l'état ; bascule vers **C** si et seulement si la cible devient cloud-native, et alors **conjointement** avec l'externalisation du store de configuration, faute de quoi le gain de scalabilité ne serait pas atteint.

9. Références

- MinIO — *Documentation* (déploiement, codage à effacement, cohérence).
- Amazon S3 — *API Reference* : `GetObject` (Range), `ListObjectsV2` , *Multipart Upload*.
- Dossier d'architecture technique — `03-dossier-architecture-technique.md` .
- Décisions d'architecture — `05-adr/0003-backend-crypto-rust-pur-hs256.md` , `05-adr/0004-image-statique-musl-distroless.md` .
- Registre des dettes techniques — `12-registre-dette-technique.md` (entrée D8).
- Note de cadrage — `01-note-de-cadrage.md` .